

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

PROGRAMACION I

TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas

Estudiantes

Andrea Barrios

Carlos German Zapatta

Docentes

Fecha de entrega

12/06/2026

ÍNDICE

1. Introducción	3
2. Marco Teórico	4
2.1 Listas	4
2.2 Diccionarios	4
2.3 Funciones	4
2.4 Estructuras Condicionales	4
2.5 Ordenamientos	4
2.6 Estadísticas Básicas	5
2.7 Archivos CSV	5
3. Decisiones Técnicas y Arquitectura	5
3.1 Descripción del Sistema	5
3.2 Diagrama de Flujo	6
3.3 Capturas de Ejecución	6
3.4 Manejo de Excepciones y Validaciones	7
4. Participación de los Integrantes	8
5. Dificultades y Conclusiones	8
6. Bibliografía	9
7. Enlaces del Proyecto	9

1. Introducción

En este Trabajo Práctico Integrador se nos presentó la oportunidad de aplicar de manera práctica los conocimientos adquiridos durante la cursada de Programación I mediante el desarrollo de un sistema de gestión de países en Python.

El programa fue diseñado para permitir la carga y administración de información de distintos países, brindando funciones para agregar registros, actualizar datos, realizar búsquedas, aplicar filtros, ordenar la información según diferentes criterios y generar estadísticas básicas a partir de los datos almacenados.

A lo largo del desarrollo trabajamos con herramientas y conceptos vistos en la materia, como listas, diccionarios, funciones, estructuras de control, manejo de excepciones y lectura de archivos CSV. También buscamos mantener una estructura clara y organizada para facilitar la comprensión y el mantenimiento del código.

Además de las funcionalidades solicitadas en la consigna, decidimos incorporar una opción adicional al sistema: la creación automática de un archivo CSV de prueba. Esta funcionalidad fue implementada mediante la opción 13 del menú y permite generar un conjunto de datos inicial cuando no se dispone de un archivo CSV previamente creado. Consideramos que este recurso resulta útil para realizar pruebas de funcionamiento, verificar las distintas operaciones del sistema y facilitar su utilización durante el desarrollo y la evaluación del proyecto.

La realización de este trabajo nos permitió integrar diferentes contenidos de la materia en una única aplicación, enfrentando situaciones reales de programación y fortaleciendo nuestras capacidades de análisis, organización y resolución de problemas.

2. Marco Teórico

2.1 Listas

Las listas fueron una de las estructuras principales utilizadas en este proyecto. Nos permitieron almacenar todos los países cargados en el sistema dentro de una misma colección de datos. Gracias a ellas fue posible recorrer los registros, realizar búsquedas, aplicar filtros, generar estadísticas y ordenar la información según distintos criterios.

2.2 Diccionarios

Cada país fue representado mediante un diccionario. Esta estructura nos permitió organizar la información utilizando claves descriptivas como nombre, población, superficie y continente.

El uso de diccionarios facilitó el acceso a cada dato específico de un país y permitió trabajar con la información de forma más clara y ordenada dentro de las diferentes funciones del sistema.

2.3 Funciones

Para desarrollar el programa se utilizó una estructura modular basada en funciones. Cada función fue diseñada para realizar una tarea específica, como cargar datos, agregar países, realizar búsquedas, aplicar filtros o generar estadísticas.

Esta forma de organización permitió que el código resultara más fácil de comprender, mantener y reutilizar, además de simplificar el proceso de desarrollo y prueba de cada funcionalidad.

2.4 Estructuras Condicionales

Las estructuras condicionales fueron utilizadas para controlar el flujo del programa y tomar decisiones según las acciones realizadas por el usuario.

A través de sentencias como if, elif y else se implementaron validaciones, controles de errores y la selección de las distintas opciones disponibles en el menú principal.

2.5 Estructuras Repetitivas

Las estructuras repetitivas permitieron ejecutar acciones varias veces según fuera necesario. Fueron utilizadas principalmente para mantener activo el menú principal durante toda la ejecución del programa y para recorrer las listas de países al realizar búsquedas, filtros, ordenamientos y cálculos estadísticos.

También se emplearon para repetir solicitudes de datos cuando el usuario ingresaba información incorrecta.

2.6 Ordenamientos

Los ordenamientos permiten organizar los datos siguiendo un criterio determinado. En este proyecto implementamos ordenamientos por nombre, población y superficie.

Además, se brindó la posibilidad de elegir entre orden ascendente y descendente, permitiendo visualizar la información de diferentes maneras según las necesidades del usuario.

2.7 Estadísticas Básicas

Las estadísticas permiten obtener información útil a partir de un conjunto de datos. Dentro del sistema se desarrollaron funciones capaces de identificar el país con mayor población, el país con menor población, calcular promedios y determinar la cantidad de países registrados por continente.

Estos resultados ayudan a interpretar la información almacenada de forma rápida y sencilla.

2.8 Manejo de Excepciones

Durante el desarrollo del proyecto se incorporó el manejo de excepciones para evitar que el programa se detuviera ante errores producidos por entradas incorrectas o problemas durante la lectura de archivos.

Mediante estructuras try y except fue posible controlar situaciones como el ingreso de texto cuando se esperaba un número, la carga de archivos inexistentes o la presencia de registros inválidos dentro del archivo CSV.

Gracias a estas validaciones el sistema ofrece una experiencia de uso más segura y estable.

2.9 Archivos CSV

Los archivos CSV fueron utilizados para almacenar la información de los países de forma permanente. Esto permitió guardar los datos fuera del programa y cargarlos nuevamente cuando fuera necesario.

Además de la carga de información desde archivos CSV, se incorporó una funcionalidad adicional que permite generar automáticamente un archivo de prueba con datos precargados. Esta característica fue incluida para facilitar las pruebas del sistema cuando no se dispone de un archivo CSV previamente creado, permitiendo verificar rápidamente todas las funcionalidades implementadas.

3. Decisiones Técnicas y Arquitectura

3.1 Descripción del Sistema

La aplicación permite cargar datos desde un archivo CSV, agregar nuevos países, actualizar información existente, realizar búsquedas, aplicar filtros, ordenar registros y generar estadísticas básicas. Todas estas operaciones se realizan sobre una colección de países almacenada en memoria durante la ejecución del programa.

Para el almacenamiento de la información se utilizó una lista de diccionarios. Cada diccionario representa un país y contiene los campos nombre, población, superficie y continente. Esta estructura fue elegida porque permite acceder fácilmente a los datos de cada registro y realizar operaciones como búsquedas, filtros y ordenamientos de forma sencilla.

El programa fue organizado mediante funciones independientes, asignando una responsabilidad específica a cada una de ellas. Esta decisión permitió mantener el código más ordenado, facilitar las pruebas durante el desarrollo y simplificar futuras modificaciones.

Además de las funcionalidades solicitadas en la consigna, se incorporó una opción adicional que permite generar automáticamente un archivo CSV de prueba. Esta herramienta fue desarrollada para facilitar las pruebas del sistema cuando no se dispone de un archivo CSV previamente creado, permitiendo verificar rápidamente todas las funciones implementadas.

3.2 Decisiones de Diseño

Durante el desarrollo se tomaron distintas decisiones con el objetivo de mejorar la organización y facilidad de uso del sistema.

Una de ellas fue centralizar las validaciones en funciones específicas, evitando repetir código en distintos sectores del programa. De esta manera fue posible reutilizar validaciones de texto y números en diferentes operaciones.

También se decidió implementar una función para limpiar la pantalla entre cada operación. Esto permite que la información mostrada al usuario resulte más clara y evita que la consola acumule grandes cantidades de texto durante el uso del sistema.

Para los ordenamientos se utilizó la función integrada `sorted()` de Python, ya que ofrece una solución eficiente y sencilla para organizar los registros según distintos criterios.

Por otra parte, se incorporaron controles para evitar el ingreso de datos inválidos, como campos vacíos, valores negativos o rangos inconsistentes en los filtros de búsqueda.

3.3 Diagrama de Flujo

El funcionamiento general del sistema puede representarse mediante el siguiente esquema:



3.4 Capturas de Ejecución

A continuación, se presentan capturas de pantalla que muestran el funcionamiento de las principales características implementadas en el sistema.

Menú principal del sistema.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TE
-----
Sistema de Gestión de Países

1. Cargar Países desde CSV
2. Mostrar Países
3. Agregar País
4. Actualizar País
5. Buscar País
6. Filtrar por Continente
7. Filtrar por Población
8. Filtrar por Superficie
9. Ordenar por Nombre
10. Ordenar por Población
11. Ordenar por Superficie
12. Mostrar Estadísticas

-----
13. Crear CSV de Prueba
-----

14. Salir

Seleccione una opción:
```

Se muestra el menú principal desde el cual el usuario puede acceder a todas las funcionalidades implementadas.

Carga de países desde archivo CSV.


```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

Países cargados correctamente.

Presione Enter para continuar...█
```

Se presenta el proceso de carga de información desde un archivo CSV para incorporar registros al sistema.

Agregado de un nuevo país.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

Ingrese el nombre del país: Portugal
Ingrese la población: 23162312
Ingrese la superficie: 123123
Ingrese el continente: Europa

País agregado correctamente.

Presione Enter para continuar...█
```

Aquí se muestra el procedimiento de alta de un nuevo país y las validaciones realizadas sobre los datos ingresados.

Búsqueda de países por nombre.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Ingrese el nombre o parte del nombre: portu

Nombre        Población        Superficie        Continente
Portugal      23162312         123123            Europa

Presione Enter para continuar...█
```

En esta imagen se ejemplifica la búsqueda de registros mediante coincidencia total o parcial del nombre ingresado.

Filtrado de países por continente.

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Ingrese continente: Europa

Nombre	Población	Superficie	Continente
Alemania	83149300	357022	Europa
España	47431256	505990	Europa
Portugal	23162312	123123	Europa

Presione Enter para continuar...

Aquí se puede ver la utilización de filtros para visualizar únicamente los países pertenecientes a un continente específico.

Ordenamiento de países por nombre.

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Nombre

Población

Superficie

Continente

Alemania

83149300

357022

Europa

Argentina

45376763

2780400

América

Australia

25690000

7692024

Oceanía

Brasil

213993437

8515767

América

Canadá

38008005

9984670

América

China

1412600000

9596961

Asia

Egipto

109262178

1002450

África

España

47431256

505990

Europa

Japón

125800000

377975

Asia

México

128932753

1964375

América

Portugal

23162312

123123

Europa

Presione Enter para continuar...

Aquí se presenta el resultado de ordenar los registros según el nombre de cada país.

Estadísticas generadas por el sistema.

```
===== ESTADÍSTICAS =====

País con mayor población: China (1412600000)
País con menor población: Portugal (23162312)
Promedio de población: 204855091.27
Promedio de superficie: 3900068.82

Cantidad de países por continente:
América: 4
Asia: 2
Europa: 3
Oceanía: 1
África: 1

Presione Enter para continuar...█
```

Aquí se muestran las estadísticas calculadas automáticamente a partir de los datos almacenados.

3.5 Manejo de Excepciones y Validaciones

Con el objetivo de mejorar la robustez del sistema se implementó el manejo de excepciones mediante estructuras try/except.

Estas estructuras permiten controlar errores producidos por entradas incorrectas del usuario o problemas relacionados con la lectura de archivos.

Entre las validaciones implementadas se encuentran:

- Verificación de campos obligatorios.
- Control de valores numéricos negativos.
- Validación de opciones del menú.
- Detección de archivos CSV inexistentes.
- Verificación de registros inválidos dentro del archivo CSV.
- Control de rangos válidos en los filtros de población y superficie.

Gracias a estas validaciones el programa puede continuar funcionando correctamente ante situaciones inesperadas, brindando mensajes claros al usuario y evitando interrupciones durante la ejecución.

4. Participación de los Integrantes

Para la realización de este proyecto nos repartimos las distintas tareas según las necesidades que fueron surgiendo durante el desarrollo. A medida que avanzábamos, íbamos revisando juntos las funcionalidades implementadas y proponiendo mejoras para lograr un sistema más completo y fácil de utilizar.

Una de las decisiones que tomamos fue organizar el programa utilizando funciones independientes para cada operación, lo que nos permitió trabajar de manera más ordenada y simplificó tanto las pruebas como las correcciones realizadas durante el desarrollo.

También decidimos incorporar una funcionalidad adicional que no era obligatoria en la consigna: la opción 13 del menú, destinada a generar automáticamente un archivo CSV de prueba. Si bien el proyecto cuenta con su archivo CSV incluido, tal como se solicita en la consigna, decidimos mantener esta funcionalidad porque resultó de gran ayuda durante el desarrollo y las pruebas del sistema. Gracias a ella fue posible generar rápidamente datos de ejemplo para verificar el correcto funcionamiento de las distintas opciones del programa.

Consideramos que esta herramienta constituye una evidencia del trabajo realizado y de las decisiones tomadas durante el desarrollo, ya que surgió como una necesidad práctica mientras construíamos y probábamos el sistema. Por este motivo decidimos conservarla en la versión final, tanto por su utilidad como por el valor que aporta al proyecto.

A lo largo del proyecto ambos participamos en el análisis de los requerimientos, la implementación de mejoras, la realización de pruebas y la elaboración de la documentación. Las decisiones más importantes relacionadas con el diseño y funcionamiento del sistema fueron discutidas y acordadas en conjunto.

5. Dificultades y Conclusiones

Durante el desarrollo del proyecto surgieron diferentes desafíos relacionados con la lectura de archivos CSV, la validación de datos ingresados por el usuario y la implementación de búsquedas, filtros y ordenamientos sobre los registros almacenados. Una de las principales dificultades fue asegurar que los datos ingresados fueran válidos, evitando campos vacíos o valores numéricos incorrectos. Para resolver estos inconvenientes se incorporaron controles y validaciones que permitieron mejorar la robustez del sistema. Asimismo, fue necesario organizar el código mediante funciones específicas para cada tarea, logrando una estructura más clara, modular y fácil de mantener.

Como conclusión, este trabajo permitió aplicar de manera práctica los conceptos estudiados en Programación I, especialmente el uso de listas, diccionarios, funciones, estructuras condicionales, estructuras repetitivas y manejo de archivos CSV. Además,

se fortalecieron las habilidades de análisis, resolución de problemas y desarrollo de programas completos.

6. Bibliografía

Python Software Foundation. Python Documentation.
Disponible en: <https://docs.python.org>

Python Software Foundation. Tutorial de Python.
Disponible en: <https://docs.python.org/3/tutorial>

Material teórico y apuntes de la asignatura Programación I.

Documentación oficial del módulo CSV de Python.
Disponible en: <https://docs.python.org/3/library/csv.html>

7. Enlaces del proyecto:

Repositorio GitHub:
<https://github.com/cgzapatta/TPI-Programacion-1.git>

Video explicativo: